INKA

IGISORO | DAME | BIYARI

PRODUCTION READINESS AUDIT

AND BINDING IMPLEMENTATION ORDER

CURRENT VERDICT NOT READY for a genuine public, multi-user online service. The supplied repository is a front-end prototype with simulated data and no production backend. Do not describe it as live multiplayer until the release gates in this document have been passed.

Prepared from the source archive supplied on 29 October 2026

Purpose: instruct an engineering agent to fix the observed defects, implement the missing production capabilities, and prove readiness with repeatable tests - not merely return another report.

1. Executive decision and audit boundaries

DECISION A static preview may be published only if it is clearly marked as a prototype. A real public service where independent people can register, challenge one another, play synchronized games, comment, and keep reliable records requires the backend and safeguards described below.

What was inspected

- The uploaded ZIP contains 38 files, including the React/Vite application, game components, context state, localization dictionaries, unit tests, deployment configuration, and assets under public/assets/aistudio/.
- The README files themselves describe INKA as a front-end prototype and explicitly say that production authentication, a database, live matchmaking, and real-time multiplayer are not implemented.
- The inspected src/ code contains hard-coded player profiles, match samples, comments, history, and initial account-like state. The only browser storage use found in the application source is localStorage for the language preference.
- No application API client, database SDK, authentication integration, server entry point, WebSocket, or EventSource implementation was found in the supplied application source.

What this audit does not claim

- No deployed website URL was supplied for live browser testing. Hosting, DNS, TLS, headers, production logs, and the actual deployed build have not been inspected.
- Dependency installation did not complete in this audit environment. A global TypeScript check also stopped because the project dependency type `vite/client` was unavailable. Therefore lint, test, and production-build success are NOT verified.

NO FABRICATION RULE Treat the statements above as the observed facts. Do not invent users, test results, server integrations, security certifications, traffic figures, legal compliance, or successful deployments. Report each unverified item as unverified until it has evidence.

Release classification

Target	Current status	Decision
Design preview / local prototype	Prototype behavior and sample data are present; build not verified here.	May be used for a clearly labeled demonstration after a successful build and smoke test.
Public single-device demo	No production account system is present.	Possible only with explicit demo labeling, no misleading live-player claims, and reviewed public assets.
Real public multiplayer service	No production backend, identity, persistence, or live synchronization found.	BLOCKED until all P0 work and acceptance gates pass.

2. Findings ranked by release risk

P0 means a blocker to representing INKA as a real online service. P1 means required before broad public release. P2 means a quality improvement that must be scheduled and verified; no priority label is permission to ship a known defect.

Priority	Finding	Required outcome
P0	No production backend or identity layer	Implement secure accounts or a deliberately limited guest model, durable storage, authenticated APIs, and documented deployment configuration.
P0	Demo records look like real people and live activity	Remove sample identities from production, or isolate them behind an unmistakable demo mode. Never present generated counts or profiles as actual activity.
P0	No shared authoritative game state	Persist matches and validate every move/result on the server. Two independent devices must see the same accepted moves, turns, clocks, and outcome.
P0	Challenges, scheduling, and spectator activity are mostly local simulations	Implement the stated workflows end to end, or hide/rename them so the interface does not promise an action that did not happen.
P0	Game rules and outcomes are not covered by gameplay tests	Verify each ruleset, correct legal-move and end-state handling, and add extensive tests before scores or rankings are trusted.
P0	Security, privacy, abuse handling, and operational recovery are not implemented for a real service	Add server-side access controls, input/rate limits, reporting/moderation, privacy notices, backups, monitoring, and incident procedures.
P1	Several visible strings and metadata claims are inconsistent with the implementation	Complete language coverage, remove unsupported marketing/AI capability claims, and verify public metadata.
P1	Reproducible build and end-to-end release evidence are missing	Commit a lockfile, use deterministic CI installation, run all checks, and provide test logs and staging smoke-test evidence.

Binding instruction to the engineering agent

Do not stop after presenting a plan, listing errors, or changing only the UI. Work through the items in priority order, edit the repository, install and configure real integrations where possible, implement tests, run them, fix failures, and deliver the complete updated project. Do not replace missing backend work with mock data or fake success messages.

CREDENTIALS RULE Never fabricate API keys, SMTP credentials, OAuth secrets, domain ownership, database URLs, or deployment success. If a secret must be supplied by the project owner, implement environment-variable validation, document the exact variable and setup steps, and identify that remaining external configuration precisely.

3. Defects observed in the supplied source

Fix these concrete issues in addition to building the missing service layer. Keep the existing INKA identity, the three games, the trilingual goal, and the no-wagering policy.

State and user identity

- `src/context/PlatformContext.tsx` defines hard-coded community players, live matches, spectator comments, match history, and a sample current user. These are demo fixtures, not real accounts.
- `startMatch` creates a local match and assigns a random spectator count, but does not add the match to `liveMatches`; the spectator list therefore cannot be a truthful live feed of matches started through the lobby.
- Profile edits, availability, honor points, match history, and results are in React state and are not saved to a durable server database. They disappear when the app state resets or are not shared with another device.
- `triggerMockIncomingChallenge` always creates a fixed sample challenge. Keep such controls only in a visible demo/development mode; production alerts must come from real authenticated events.

Workflow defects

- `src/components/modals/ChallengeModal.tsx`: the friendly-message input is updated in local state, but `handleSubmit` calls `onStart(selectedGame, timeControl)` without passing the message. The typed message is discarded.
- `src/components/views/SchedulingView.tsx`: confirming a challenge only displays a confirmation notice. It does not persist a schedule, send an invitation, record an event, or verify the other player accepted it.
- `src/components/views/SpectatorView.tsx`: the match is copied into local component state. Adding a comment updates context data, but the selected copy can remain stale, so the new comment may not appear immediately. The view also assumes `liveMatches[0]` exists and has no empty/loading/error state.
- `src/components/views/SpectatorView.tsx` shows a symbolic match stage rather than a real live board stream; its match metadata is sample data. Build synchronized spectator state or mark the panel as a demo preview.

Game state and results

- `src/components/games/DameGame.tsx`: a human player can be offered ordinary moves even when a capture is available; the opponent simulation prioritizes captures but the human move path does not enforce that priority. A capture does not implement a continuing multi-capture sequence, and the source does not conclude the game when a side has no legal move.
- `src/components/games/GameHud.tsx`: the draw flow displays a message that a draw was mutually agreed and then concludes locally; there is no second-player offer/acceptance transaction. Define and consistently record win, loss, draw, abandoned, and no-contest outcomes.
- `src/components/games/BiyariGame.tsx`: several user-visible status messages and the victory-modal label are hard-coded in Kinyarwanda. They bypass the RW/EN/FR translation dictionaries.

4. Additional source findings and required cleanup

- `src/App.tsx` passes `onSelectGame={(g) => {}}` to `LobbyView`; the callback is a no-op. Remove the dead interface or wire it to a real action. No visible control should silently do nothing.
- `metadata.json` claims INKA is Rwanda's "premier" community platform and lists a server-side Gemini API capability. The inspected app source contains no corresponding backend integration, and no evidence was supplied to substantiate the superlative. Remove or verify these claims before release.
- The archive has no `package-lock.json`. The CI workflow uses `npm install`. Add and commit a lockfile, use `npm ci` in CI/deployment, pin the supported Node version, and fix any dependency/audit findings rather than blindly applying updates.
- The `.env.example` lists `APP_URL` and `VITE_LAUNCH_PHASE`, but the supplied application source does not appear to use them. Either wire them into clearly defined behavior or remove them. Never put a secret in a `VITE_` variable because frontend values are exposed to the browser.
- `public/assets/aistudio/` contains two PDFs and a Python file. Vite serves files under `public/` as public static assets. Confirm that every asset is intentionally public; move internal specifications or working files outside the public directory.
- The current tests cover two utility test files: scoring formulas and translation-key parity. They do not verify browser workflows, actual board legality, backend authorization, persistence, multiplayer synchronization, accessibility, or deployment behavior.

5. Minimum target architecture

Keep the current Vite/React frontend if useful, but add a real service boundary. Choose one maintainable backend stack and implement it end to end; do not leave stubs, fake API responses, or TODO-only routes.

Layer	Minimum responsibilities
Identity	Sign-up, sign-in, sign-out, session lifecycle, secure password recovery if passwords are used, account state, and server-enforced authorization.
Database	Durable users/profiles, challenges, match records, ordered moves/events, results, honors, schedules, comments, reports/blocks, and necessary audit records. Add migrations, constraints, indexes, and backup/restore procedures.
Game service	Authoritative match status, whose turn it is, legal-move validation, clocks, turn/move versioning, idempotency, conclusion rules, and server-calculated result/honor changes.
Realtime layer	Deliver accepted moves and status updates to both players and spectators; support reconnect/rejoin, duplicate-event protection, stale-state handling, and disconnect/time-out policy.
Moderation and support	Report/block flows, rate limits, spam controls, content handling, admin permissions, and auditable actions. Do not expose privileged controls to ordinary users.
Operations	Environment validation, structured logs, monitoring/alerts, health checks, backups, restore tests, error tracking, and a documented rollback procedure.

SERVER AUTHORITY The browser must not be trusted to decide the winner, grant honor points, set spectator counts, change another user's profile, approve a challenge, or write arbitrary match results. Validate these actions on the server and authorize every record by its owner and match role.

6. Implement each user workflow end to end

Accounts and profiles

- A person must be able to create an account or enter an explicitly limited guest/demo mode; sign in/out; recover access where applicable; view and edit their own profile; and return later to find the same saved data.
- Persist profile, preferred language, availability, preferences, match history, and earned honors according to a documented data model. Handle loading, validation errors, network failures, expired sessions, and successful saves clearly.
- Production profile lists must contain only real opted-in records returned by the backend. Do not seed fictional people into a production community roster.

Challenges and scheduling

- A challenge must be a persisted record with sender, recipient, selected game, time control, optional bounded-length message, creation time, expiration, and status: pending, accepted, declined, expired, or cancelled.
- The sender must receive a truthful status. The recipient can accept or decline. A real match starts only after the required acceptance flow. Handle duplicate requests, self-challenges, unavailable/blocked users, and stale requests.
- If scheduling is offered, persist the selected time and time zone, check availability/conflicts, notify both parties, and show cancellation/update status. Otherwise remove the feature or clearly call it a UI preview.

Match play, spectator arena, and chat

- Create a match record when it is actually accepted. Every accepted move must receive an authoritative server response with a sequence/version number. Do not trust a client-supplied score or board state.
- The spectator arena must query real public matches, handle the zero-match state, and subscribe to actual match updates. The displayed board, turn, score, elapsed time, and viewer count must not be made up.
- Persist comments, validate length/content, attribute the authenticated author, apply rate limits, support reporting/blocking, and refresh the selected comment feed from the canonical match record.

Awards and history

- Calculate honors and ratings from server-validated match outcomes only. Define how draws, resignations, timeouts, disconnects, abandoned matches, and invalid/forfeited matches affect each statistic.
- Separate honor points, rating, sportsmanship level, and badges into clearly defined concepts. Avoid showing badges as earned until the server verifies their criteria.

ACCEPTANCE TEST Use two separate devices or browser profiles. A challenge from device A must appear on B; B accepts it; both see the same match ID and ordered turns; moves and timer state stay synchronized; a disconnect/reload restores the correct match; and the final result and history are identical on both devices.

7. Verify the actual rules of all three games

Do not guess at traditional rules. Obtain and record a credible rules reference or review by a qualified Rwandan game/cultural expert. Document the exact variant INKA implements, the source, and any intentional deviations. Do not describe an unverified custom variant as canonical.

Dame / checkers

- Verify initial placement, direction of ordinary pieces, king movement, captures, whether captures are mandatory, whether the longest capture or multiple jumps are required in this variant, king promotion timing, draw rules, and the no-legal-move end condition.
- Enforce the same validated rules for both the human and computer opponent. The UI must not offer a move that the game engine will later reject. Ensure game state cannot be moved after match conclusion.

Igisoro

- Verify the 4-by-8 layout, starting seed distribution, sowing path/direction, extra-turn rules, capture/"gusiba" conditions, empty-pit handling, player-side ownership, and end-of-game rules against the selected Rwandan variant.
- Add invariant tests for valid pit counts and total seed conservation where the chosen rule requires it. Test blocked/empty sides, long sowing paths, capture boundaries, invalid selections, and termination without infinite loops.

Biyari / eight-ball

- Document the actual rule set. Test rack layout and groups; legal break; first-contact and rail requirements; scratches/fouls; ball-in-hand; group assignment; pocket calls; early or wrong-pocket 8-ball; legal 8-ball victory; and game reset.
- Test the custom canvas physics on low-end Android devices, different viewport sizes, and touch input. Physics should not determine a legal win without the game-rules service confirming the complete shot outcome.

Shared requirements for every game

- Create pure game-engine functions with unit tests separate from UI drawing. Add regression fixtures for legal/illegal states, edge cases, end states, and adversarial client requests.
- Test time controls, pause/resume, resign, draw proposal/acceptance, voluntary stop, disconnect, reconnect, stale moves, and duplicate move submissions. Use a single result/outcome model shared by server, history, ratings, and UI.
- Make the rules/help drawer describe the implemented rule set, not merely an aspiration. Have local-language terminology reviewed by fluent speakers and game experts.

NO UNVERIFIED RULE CLAIMS If a rule cannot be confirmed, flag it as an open decision and do not silently invent a rule. Complete the rule verification before accepting competitive ratings or presenting the game as a faithful official version.

8. Security, privacy, and community safety

Required server-side controls

- Authorize every request on the server. A user can edit only their own profile; only eligible participants can submit match actions; only authorized roles can moderate or administer records.
- Validate and bound all user-controlled values on both client and server: names, profile text, messages, comments, game identifiers, move payloads, IDs, timestamps, and requested state transitions.
- Apply rate limits and abuse controls to sign-in/recovery, challenges, comments, moves, reports, and other public endpoints. Use safe error messages and prevent enumeration of accounts where appropriate.
- Use secure sessions appropriate to the chosen architecture, secure password hashing if passwords are stored, transport security, strict CORS policy, security headers, and CSRF protection where cookie-based sessions require it. Do not store privileged secrets in frontend code.
- Review database row-level policies or equivalent authorization rules. Test that changing an object ID cannot expose another person's data or alter another match.
- Scan dependencies and repository history for known vulnerabilities and accidentally committed secrets. Do not blindly apply breaking updates; resolve and document each relevant finding.

Privacy and abuse readiness

- Before collecting real account or location data, publish readable Privacy Notice, Terms of Use, community rules, contact/support information, data-retention/deletion process, and any cookie/analytics disclosure that applies. Have the policy reviewed for Rwanda and any other intended markets.
- Collect only the data needed for the service. Explain what is public on a profile, provide privacy controls, and ensure precise location or contact details are not exposed accidentally.
- Provide report and block controls, a practical moderation workflow, spam/harassment response, an escalation owner, and an audit trail for administrative actions. Decide age eligibility and age-appropriate safeguards before public launch.
- Keep the no-wagering promise: no betting, stakes, wallet, cash-out, paid outcomes, or money-transfer features. Do not add them as a shortcut to monetization.

Operational resilience

- Separate development, staging, and production settings. Validate required environment variables at startup and fail safely when a required integration is missing.
- Configure monitored errors/uptime, actionable alerts, database backups, documented retention, and a successful restore test. Protect logs from passwords, tokens, and unnecessary personal data.
- Document who can administer the service, rotate secrets, investigate abuse, deploy a fix, and roll back a faulty release. Never grant broad third-party access without a defined need and least-privilege permissions.

9. Localization, accessibility, mobile quality, and public content

Three-language parity

- Every visible user-facing string must come from the RW/EN/FR dictionaries or a structured translation resource, including canvas/game status, error states, modal text, aria labels, confirmation messages, empty states, and notifications.
- Fix hard-coded Kinyarwanda text in `BiyariGame.tsx`. Search the entire repository for remaining hard-coded user-facing strings, not just translation keys.
- Keep key-parity tests, but add rendered-UI tests in every language and human review by fluent speakers. Equal dictionary keys do not prove natural translation, correctness, or that all visible text is translated.

Accessibility and device support

- Associate form labels with inputs; use fieldset/legend semantics for button groups; provide visible focus indicators, keyboard support, focus trapping/restoration for dialogs, Escape handling where safe, and useful screen-reader announcements.
- Make all essential actions usable on Android Chrome by touch, without requiring a mouse or desktop-size screen. Verify portrait/landscape changes, zoom, reduced-motion preference, contrast, and small-screen modal overflow.
- Provide an accessible textual alternative for the Biyari canvas and clear labels for game-board controls. Do not convey a legal move or result using color alone.
- Create loading, empty, offline, validation, and retry states. A slow or interrupted network must not leave the user believing a challenge, save, move, or comment succeeded when the server did not confirm it.

SEO and content accuracy

- Remove unsupported claims such as "Rwanda's premier" unless substantiated. Remove or implement the metadata claim about server-side Gemini capability; metadata must describe the real shipped service.
- Set production title, description, canonical URL, social-preview image, favicon, robots/sitemap policy, and localized metadata deliberately. Do not expose private specifications or working files in the public static-asset directory.
- Use external fonts only after considering privacy, caching, low-bandwidth performance, and fallback behavior. Avoid blocking first paint on nonessential resources.

USER TRUST A calm, accurate message that says which features are live is better than a polished screen that suggests nonexistent opponents, viewers, rankings, or challenges. Prototype labeling must be removed only when the matching feature has actually passed its acceptance tests.

10. Reproducible build and deployment requirements

Repository and CI

- Add a committed `package-lock.json` that matches the chosen package manager and dependency versions. Use `npm ci` in CI and repeatable build environments.
- Choose and pin a supported Node.js version compatible with the installed Vite and TypeScript versions; declare it in project configuration and use the same version in CI, staging, and production.
- Make the pipeline fail on type errors, lint failures, unit/integration/e2e test failures, production build errors, and applicable security checks. Add tests instead of weakening checks to obtain a green build.
- Keep CI permissions least-privilege. Do not expose secrets to untrusted pull requests. Add dependency/secret scanning and a documented process for triaging findings.

Production configuration

- Implement and document every required server URL, database setting, authentication callback, email/recovery setting, realtime service, and allowed-origin setting. Validate configuration at startup and fail clearly rather than silently reverting to demo behavior.
- Review `vercel.json` before adding backend routes. The current catch-all rewrite sends unmatched paths to `/index.html`; ensure real `/api` endpoints or external backend routes are not swallowed by the frontend fallback.
- Use a staging deployment first. Verify HTTPS, custom domain, redirects, deep links, static asset access, API/realtime connectivity, CORS/cookies, error pages, and security headers from a real browser.
- Check which files in `public/assets/aistudio/` are meant for public downloads. Move internal documentation, scripts, and working material to a non-public location if they are not intended for visitors.

Release and recovery

- Run database migrations safely, test backup restoration, document rollback steps, and monitor application errors and backend health after release.
- Test the published build on a real Android phone and at least two independent browser sessions on different networks when possible. Record device/browser, build identifier, test time, and outcome.
- Set a realistic initial concurrency/performance target with the project owner; load-test to that declared target and report measured results. Do not invent a capacity claim.

Required commands in a clean environment

```
npm ci
npm run lint
npm test
npm run build
npm run preview (then execute the browser smoke-test checklist)
```

EVIDENCE REQUIRED Publish the actual command output and test report with the delivery. Never write "all tests pass" unless those exact checks ran successfully against the final commit/build.

11. Mandatory verification matrix

The implementation is not complete until each applicable row passes and evidence is recorded. Automated tests do not replace rule review, privacy review, or real-device smoke testing.

Area	Required scenarios	Pass condition
Build and types	`npm ci`; type-check/lint; test; production build	All commands exit successfully on the final committed code with a reproducible lockfile.
Identity and persistence	Create/sign in/out; invalid credentials; recovery; edit profile; reload; second device	Correct user is authenticated; data persists; unauthorized records are inaccessible.
Challenges	Send, accept, decline, expire, cancel; repeat click; blocked/unavailable/self target	One valid persisted state transition; no duplicate or false success; both users see consistent status.
Live match	Two browsers, concurrent moves, duplicate/out-of-order moves, pause, timeouts, reload, network drop	Only legal moves are accepted; state and clock converge; rejoin resumes canonical state; result is single and consistent.
Dame	Opening position; legal/illegal move; capture priority; chained capture if required; king; no legal moves; draw/end state	Matches the documented rule variant; no invalid move or stuck nonterminal state.
Igisoro	Seed conservation; each sow path; capture; edge/empty pits; extra turns if applicable; no-move/endgame	All outcomes match the documented variant; engine invariants hold; no impossible seed counts.
Biyari	Break, groups, foul, scratch/ball-in-hand, wrong/early 8-ball, called pocket, touch/resize, reset	Physics and rule engine agree; invalid wins are rejected; interactions work across target devices.
Spectator and comments	Zero matches; one/many active matches; stream update; add comment; malicious/oversized input; report/block	Correct empty state; feed updates from canonical data; input limits and moderation work.
Scores and history	Win/loss/draw/resign/timeout/disconnect/ abandon; replay/duplicate conclusion	One authoritative outcome; correct history, ranking, honors, and badge criteria; no double-award.
Localization	All main flows in RW, EN, FR; validation/errors; modal and game messages; metadata	No untranslated visible strings; placeholders preserved; reviewed language is natural and accurate.
Security/privacy	Cross-user access attempts; invalid state transitions; rate limit; secret scan; report/delete requests	Unauthorized operations fail; private data stays private; documented safeguards work.
Mobile/accessibility	Android Chrome, narrow viewport, landscape, touch, keyboard/screen reader, reduced motion, slow network	No blocking layout/input defects; essential workflows remain operable and status feedback is honest.
Operations	Staging smoke test; monitoring/alerts; database backup and restore; rollback drill	The operator can detect failure, restore data, and safely roll back the release.

PASS/FAIL DISCIPLINE Mark a row PASS only with evidence. Mark it FAIL when a test fails and BLOCKED when credentials, a domain, third-party access, or an external decision is genuinely missing. Never convert BLOCKED into PASS by substituting mock behavior.

12. Definition of done and final delivery contract

Go-live gates - all are mandatory

- [ ] The production frontend and backend are deployed to the intended HTTPS domain, and the operator can identify the final commit/build.
- [ ] Real authentication/guest-mode boundaries, durable database storage, authorization rules, realtime match state, and server-authoritative results are implemented and tested.
- [ ] No sample profile, sample match, fake spectator count, fabricated history, placeholder notification, or mock response is exposed as real production activity.
- [ ] The three game rules are documented and reviewed; move validation and outcome tests pass; no known blocker remains in match state or scoring.
- [ ] Challenge, scheduling, spectator, comments, profile, honor, and history flows do what their UI claims they do, or the unsupported flow is removed/clearly disabled.
- [ ] Privacy, terms, support, moderation/report/block, secure configuration, backups/restore, monitoring, and release rollback are in place for the intended launch scope.
- [ ] All visible text is covered by RW/EN/FR localization; mobile/accessibility smoke tests pass; public metadata and static assets are accurate.
- [ ] Clean install, type-check/lint, automated tests, production build, end-to-end checks, staging smoke tests, and applicable security scans pass on the final code.

Required handover package

- Complete updated source repository or one complete ZIP. Do not deliver only changed files, screenshots, an incomplete excerpt, or a plan.
- Database migrations/schema, backend functions/services, frontend code, tests, CI configuration, deployment configuration, and a safe `.env.example` with no real secrets.
- README in clear English: architecture, prerequisites, exact setup commands, environment variables, local run, tests, deployment, migrations, backups, recovery, and how to tell demo mode from production mode.
- A change log naming major files/areas changed; exact commands executed; real test outputs; deployment URL/build identifier if deployed; and a short list of any items genuinely blocked by external credentials or decisions.

Final instruction to the engineering agent

IMPLEMENT, TEST, DELIVER Take ownership of the complete scope in this order: fix the observed defects; implement the missing backend and durable workflows; validate game rules; add security, privacy, moderation, localization, accessibility, and operations; run the full verification matrix; fix failures; then deliver the complete project and evidence. Do not stop at an audit, UI-only patch, scaffold, mock API, or partially completed ZIP.

Preserve INKA's intended identity: a Rwandan community skill-games platform for Igisoro, Dame, and Biyari; respectful play; no financial wagering; and honest communication about what is actually working.

Source files that require focused review

src/context/PlatformContext.tsx; src/App.tsx; src/components/modals/ChallengeModal.tsx; src/components/modals/IncomingCallModal.tsx; src/components/views/LobbyView.tsx; ProfilesView.tsx; SchedulingView.tsx; SpectatorView.tsx; HonorsView.tsx; src/components/games/DameGame.tsx; IgisoroGame.tsx; BiyariGame.tsx; GameHud.tsx; src/utils/matchScoring.ts; src/utils/i18n.ts and its tests; package.json; .github/workflows/verify-inka.yml; vercel.json; index.html; metadata.json; and the public asset directory.